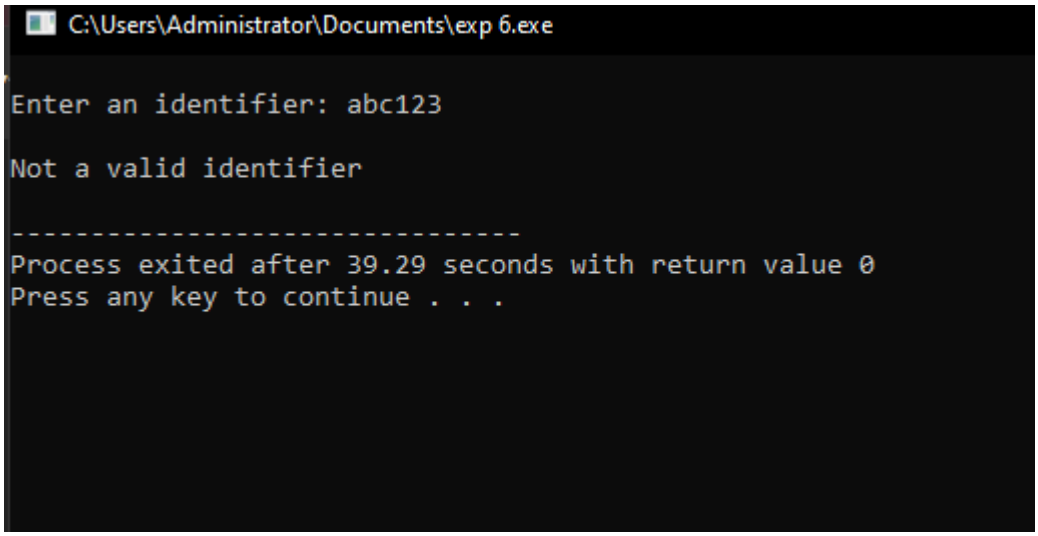
6. Develop a lexical Analyzer to test whether a given identifier is valid or not using C.

```
#include <stdio.h>
#include <ctype.h>
int main() {
 char a[10];
 int flag = 1, i = 1;
 printf("\nEnter an identifier: ");
 fgets(a, sizeof(a), stdin);
 if (isalpha(a[0])) {
 while (a[i] != '\0') {
 if (!isdigit(a[i]) && !isalpha(a[i])) {
 flag = 0;
 break;
 }
 i++;
 }
 } else {
 flag = 0;
 }
 if (flag == 1) {
 printf("\nValid identifier\n");
 } else {
 printf("\nNot a valid identifier\n");
 }
 return 0;
}
```

OUTPUT:



7. Write a C program to find FIRST( ) - predictive parser for the given grammar S → AaAb / BbBa A → ∈ B → ∈

```
#include<stdio.h>
#include<ctype.h>
void FIRST(char[],char );
void addToResultSet(char[],char);
int numOfProductions;
char productionSet[10][10];
int main()
{
 int i;
 char choice;
 char c;
 char result[20];
 printf("How many number of productions ? :");
 scanf(" %d",&numOfProductions);
 for(i=0;i<numOfProductions;i++)//read production string eg: E=E+T
 {
```

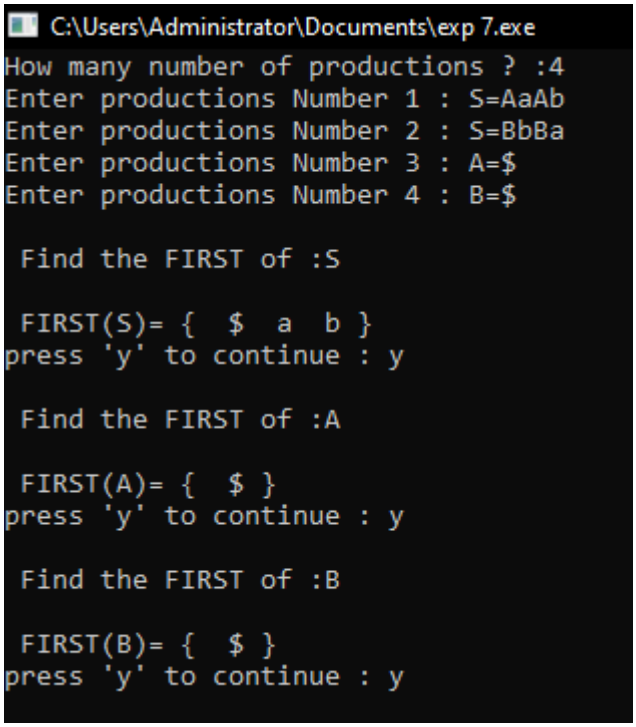
```
 printf("Enter productions Number %d : ",i+1);
 scanf(" %s",productionSet[i]);
 }
 do
 {
 printf("\n Find the FIRST of :");
 scanf(" %c",&c);
 FIRST(result,c); //Compute FIRST; Get Answer in 'result' array
 printf("\n FIRST(%c)= { ",c);
 for(i=0;result[i]!='\0';i++)
 printf(" %c ",result[i]); //Display result
 printf("}\n");
 printf("press 'y' to continue : ");
 scanf(" %c",&choice);
 }
 while(choice=='y'||choice =='Y');
}
/*
*Function FIRST:
*Compute the elements in FIRST(c) and write them
*in Result Array.
*/
void FIRST(char* Result,char c)
{
 int i,j,k;
 char subResult[20];
 int foundEpsilon;
 subResult[0]='\0';
```

```
Result[0]='\0';
//If X is terminal, FIRST(X) = {X}.
if(!(isupper(c)))
{
addToResultSet(Result,c);
return ;
}
//If X is non terminal
//Read each production
for(i=0;i<numOfProductions;i++)
{
//Find production with X as LHS
if(productionSet[i][0]==c)
{
//If X ? e is a production, then add e to FIRST(X).
if(productionSet[i][2]=='$') addToResultSet(Result,'$');
//If X is a non-terminal, and X ? Y1 Y2 ... Yk
//is a production, then add a to FIRST(X)
//if for some i, a is in FIRST(Yi),
//and e is in all of FIRST(Y1), ..., FIRST(Yi-1).
else
{
j=2;
while(productionSet[i][j]!='\0')
{
foundEpsilon=0;
FIRST(subResult,productionSet[i][j]);
for(k=0;subResult[k]!='\0';k++)
```

```
 addToResultSet(Result,subResult[k]);
 for(k=0;subResult[k]!='\0';k++)
 if(subResult[k]=='$')
 {
 foundEpsilon=1;
 break;
 }
 //No e found, no need to check next element
 if(!foundEpsilon)
 break;
 j++;
 }
 }
 }
}return ;
}
/* addToResultSet adds the computed
*element to result set.
*This code avoids multiple inclusion of elements
 */
void addToResultSet(char Result[],char val)
{
 int k;
 for(k=0 ;Result[k]!='\0';k++)
 if(Result[k]==val)
 return;
 Result[k]=val;
 Result[k+1]='\0';
```

}

OUTPUT:



8. Write a C program to find FOLLOW( ) - predictive parser for the given grammar S → AaAb / BbBa A → ∈ B → ∈

```
#include<stdio.h>

#include<ctype.h>

#include<string.h>

int limit, x = 0;

char production[10][10], array[10];

void find_first(char ch);

void find_follow(char ch);

void Array_Manipulation(char ch);

int main()

{

 int count;
```

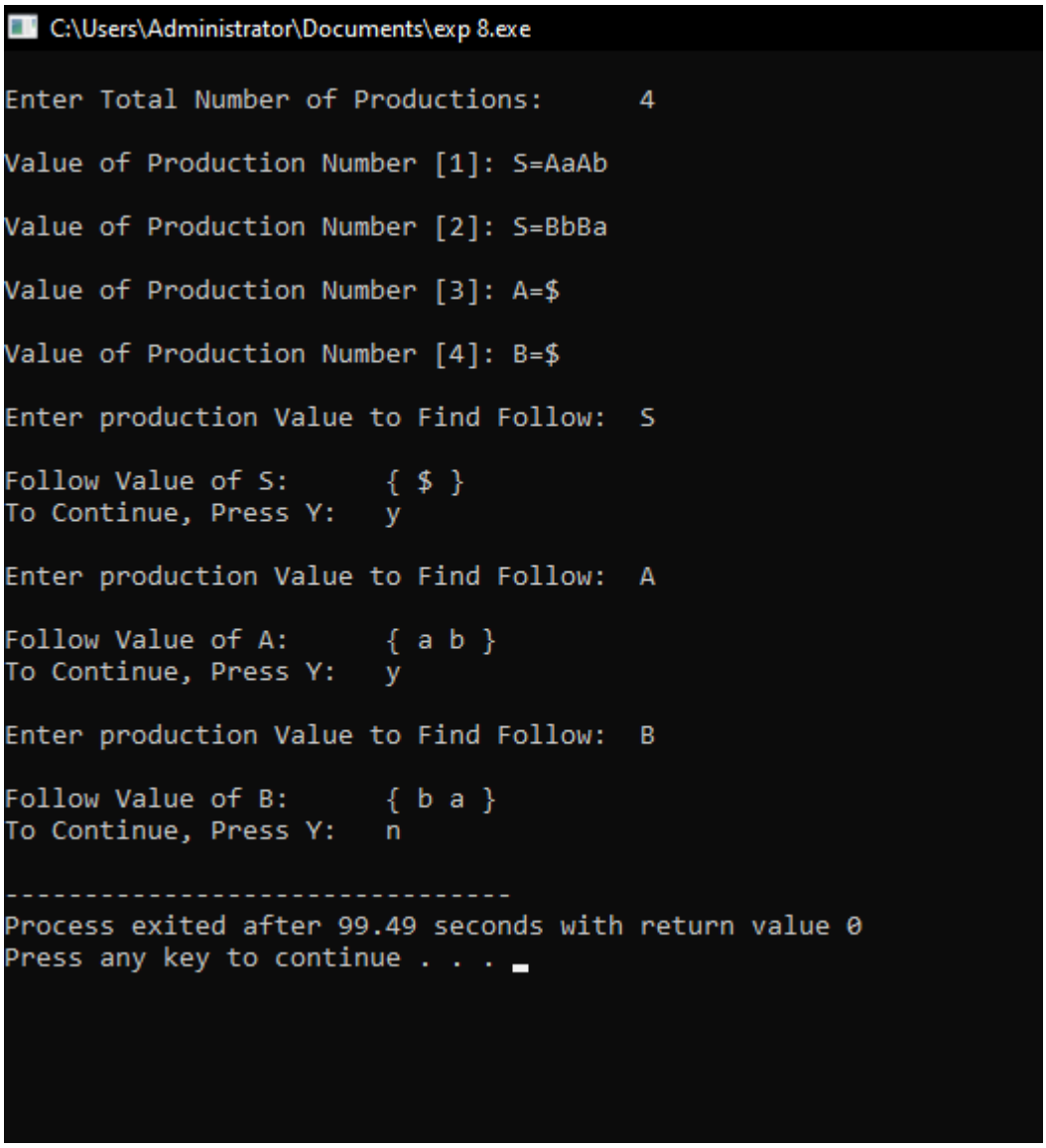
```
 char option, ch;
 printf("\nEnter Total Number of Productions:\t");
 scanf("%d", &limit);
 for(count = 0; count < limit; count++)
 {
 printf("\nValue of Production Number [%d]:\t", count + 1);
 scanf("%s", production[count]);
 }
 do
 {
 x = 0;
 printf("\nEnter production Value to Find Follow:\t");
 scanf(" %c", &ch);
 find_follow(ch);
 printf("\nFollow Value of %c:\t{ ", ch);
 for(count = 0; count < x; count++)
 {
 printf("%c ", array[count]);
 }
 printf("}\n");
 printf("To Continue, Press Y:\t");
 scanf(" %c", &option);
 }while(option == 'y' || option == 'Y');
 return 0;
}
void find_follow(char ch)
{
 int i, j;
```

```
 int length = strlen(production[i]);
 if(production[0][0] == ch)
 {
 Array_Manipulation('$');
 }
 for(i = 0; i < limit; i++)
 {
 for(j = 2; j < length; j++)
 {
 if(production[i][j] == ch)
 {
 if(production[i][j + 1] != '\0')
 {
 find_first(production[i][j + 1]);
 }
 if(production[i][j + 1] == '\0' && ch != production[i][0])
 {
 find_follow(production[i][0]);
 }
 }
 }
 }
}
void find_first(char ch)
{
 int i, k;
 if(!(isupper(ch)))
 {
```

```
 Array_Manipulation(ch);
 }
 for(k = 0; k < limit; k++)
 {
 if(production[k][0] == ch)
 {
 if(production[k][2] == '$')
 {
 find_follow(production[i][0]);
 }
 else if(islower(production[k][2]))
 {
 Array_Manipulation(production[k][2]);
 }
 else
 {
 find_first(production[k][2]);
 }
 }
 }
}
void Array_Manipulation(char ch)
{
 int count;
 for(count = 0; count <= x; count++)
 {
 if(array[count] == ch)
 {
```

```
 return;
 }
 }
 array[x++] = ch;
}
```

OUTPUT:



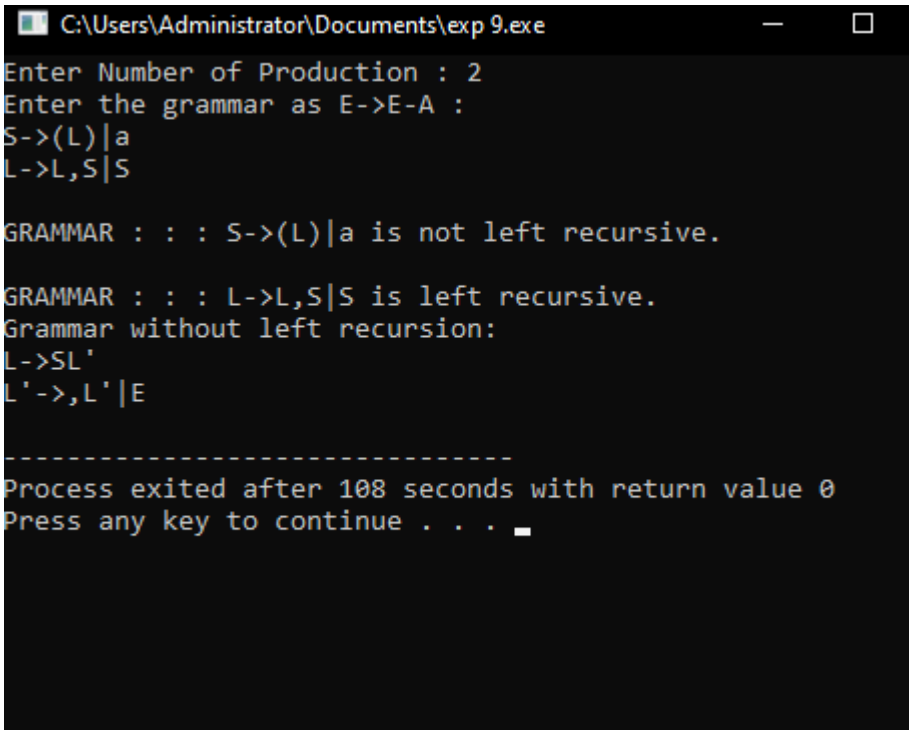
9. Implement a C program to eliminate left recursion from a given CFG. S → (L) / a L → L , S / S

#include<stdio.h>

```
#include<string.h>
#define SIZE 10
 int main () {
 char non_terminal;
 char beta,alpha;
 int num;
 char production[10][SIZE];
 int index=3; /* starting of the string following "->" */
 printf("Enter Number of Production : ");
 scanf("%d",&num);
 printf("Enter the grammar as E->E-A :\n");
 for(int i=0;i<num;i++){
 scanf("%s",production[i]);
 }
 for(int i=0;i<num;i++){
 printf("\nGRAMMAR : : : %s",production[i]);
 non_terminal=production[i][0];
 if(non_terminal==production[i][index]) {
 alpha=production[i][index+1];
 printf(" is left recursive.\n");
 while(production[i][index]!=0 && production[i][index]!='|')
 index++;
 if(production[i][index]!=0) {
 beta=production[i][index+1];
 printf("Grammar without left recursion:\n");
 printf("%c->%c%c\'",non_terminal,beta,non_terminal);
 printf("\n%c\'->%c%c\'|E\n",non_terminal,alpha,non_terminal);
 }
```

```
else
printf(" can't be reduced\n");
}
else
printf(" is not left recursive.\n");
index=3;
}
}
```

OUTPUT:



10. Implement a C program to eliminate left factoring from a given CFG. S → iEtS / iEtSeS / a E → b

```
#include<stdio.h>
#include<string.h>
int main()
{
```

```
char gram[20], part1[20], part2[20], modifiedGram[20], newGram[20];
int i, j = 0, k = 0, l = 0, pos;
// Input production
printf("Enter Production: S->");
gets(gram);
// Extract part1 and part2
for(i = 0; gram[i] != '|'; i++, j++)
part1[j] = gram[i];
part1[j] = '\0';
for(j = ++i, i = 0; gram[j] != '\0'; j++, i++)
part2[i] = gram[j];
part2[i] = '\0';
// Find common prefix
for(i = 0; part1[i] == part2[i]; i++)
{
modifiedGram[k] = part1[i];
k++;
pos = i + 1;
}
// Create modified production
modifiedGram[k] = 'X';
modifiedGram[++k] = '\0';
// Create new production
for(i = pos, j = 0; part1[i] != '\0'; i++, j++)
newGram[j] = part1[i];
newGram[j++] = '|';
for(i = pos; part2[i] != '\0'; i++, j++)
newGram[j] = part2[i];
```

```
 newGram[j] = '\0';
 // Print the result
 printf("\n S->%s", modifiedGram);
 printf("\n X->%s\n", newGram);
 return 0;
}
```

OUTPUT: